When the table being queried has poor query performance, it will be inefficient to regularly query it. Determine if you can modify the table with special-purpose indexes to speed up the query you'll need for the backfill.

On the destination side, slow updates can cause excessive lock durations and block other readers. In an upcoming chapter you'll see how to use `EXPLAIN` to analyze both the read and write.

If the queries to read and write aren't fast enough, you may want a special-built table to assist in the backfill process. Read on for more information.

Specialized Tables For Backfills

When optimizing the read side and write side queries is not enough, you might want to create a specialized table to assist in the backfill process.

The data in this table will be temporary in nature, and will duplicate the source data. A benefit of creating a special-purpose table with duplicate data is that there will be no other connections or contention for these table rows. This separate table can also be used to track the backfill state.

A downside of duplicating data is that more space is consumed.

With the intermediate table scenario, rows selected from the source table are written into a new intermediate table instead of into the target or destination table.

Since this table is designed specifically for the backfill, you'll add indexes to it or remove items from it to help speed the process up. This solution adds more effort earlier but is designed to speed up the total time of the operation.

For the specialized table, now you can start selecting what's needed and not needed. Start by reducing the columns from the source table to the bare minimum, and create the table without any indexes or constraints.

To make writes to the table even faster and avoid having this table replicated, set the table to "Unlogged".

To do that, from psql run the `SET UNLOGGED` statement for the table if it's already been created. An unlogged table does not have write protection from the Write-Ahead Log (WAL). If PostgreSQL restarts unexpectedly, you may lose data for this table. Since this table only has duplicated data though, having it unlogged is a reasonable trade-off since it could be re-created if necessary.

Once the data is populated, add indexes that are as narrow and as special-purpose as possible, closely matching the query you're using.